

Drift-Sense on AMD Ryzen

CPU-only setup and testing guide — no NVIDIA GPU required

SCOPE

This guide covers running the already-trained Drift-Sense localizer and its supporting scripts on a plain AMD Ryzen CPU, for testing and evaluation purposes only. Full training (tens of thousands of steps) is impractical on CPU and is not the goal here — see the "What NOT to expect" section.

Companion to the package README.md — read that first for full architecture and script documentation.

1. What this guide covers

Every script in this package auto-detects whether a CUDA GPU is available (`device = "cuda" if torch.cuda.is_available() else "cpu"`) and falls back to CPU automatically — no code changes are needed to run on an AMD Ryzen machine. AMD Ryzen is a CPU, not a GPU, so nothing here needs ROCm or any AMD-specific software; every command below runs on plain CPU PyTorch.

What runs comfortably on CPU

- **Single-pair prediction** (`predict.py` / `localize.py`) — one forward pass, a few seconds on CPU.
- **Dataset generation** (`generate_dataset.py`) — pure NumPy/OpenCV image synthesis, no GPU involved even normally.
- **Batch localization** over a handful to a few dozen pairs.
- **Validation report** with a small sample count (`--n-per-condition`).
- **The test suite** (`pytest`) — already CPU-friendly; that's how it runs in this guide's verification step too.

What NOT to expect

- **Full training** (`--max-steps 40000`) — this package's own GPU runs saw roughly 1.5-100+ it/s depending on how far the cosine LR schedule had progressed (early steps are fastest; later steps involve larger accumulated dataloader state). A Ryzen CPU should be treated as 10-30x slower than a consumer NVIDIA GPU for this specific workload (ResNet-18 encoder run twice per step, at 1000x1000 and 100x100 resolution) — a real 40000-step run would likely take multiple days, not the several hours quoted in the README for GPU. Use the tiny smoke-test command in Section 5 instead of real training.
- **Large validation-report runs** — keep `--n-per-condition` small (5-20) for CPU testing; the default of 50 across 4 conditions (200 samples total) will be noticeably slower than on GPU.

2. Setup

Clone/copy the `puthere/` folder onto the Ryzen machine, then from inside it:

```
python3 -m venv .venv
source .venv/bin/activate
pip install numpy opencv-python-headless pillow pytest matplotlib
pip install torch torchvision
```

Do NOT use `requirements.txt`'s pinned `torch==2.13.0+cu130/torchvision==0.28.0+cu130` lines directly — those are CUDA-specific local version labels that only resolve against PyTorch's CUDA wheel index and will fail to install (or install a GPU build that can't run) on a CPU-only machine. The plain `"pip install torch torchvision"` above pulls the standard CPU wheel from PyPI instead.

Verify the install

```
python -m pytest tests/ -q -m "not slow"
```

Should print `106 passed, 10 deselected` (exact numbers may differ slightly if the package has been updated since this guide was written) after roughly 40-90 seconds on a typical Ryzen desktop/laptop chip — noticeably slower than a GPU-equipped machine mainly because two of the test files spin up a real

ResNet-18 model, but still well within "quick sanity check" territory.

3. Predict on a single reference/search pair

Uses the shipped example images and checkpoint — no dataset generation needed first:

```
python scripts/predict.py \  
  --checkpoint checkpoints/m3_hn_r24/best.pt \  
  --reference test_examples/ref_00.png \  
  --search test_examples/search_00.png \  
  --verbose
```

Expected output (a few seconds on CPU):

```
predicted_x=338.69 predicted_y=537.09 confidence=0.2174
```

Exact numbers depend on which checkpoint you point at — checkpoints/production_v2/best.pt or checkpoints/production_v3/best.pt (if present) are the real, fully-trained checkpoints and will show much higher confidence than the intentionally under-trained m3_hn_r24 used here as a small/fast example.

4. Generate a small test dataset

Pure CPU image synthesis (NumPy/OpenCV) — no GPU involved even on the reference NVIDIA setup, so this step runs at normal speed:

```
python generate_dataset.py \  
  --split test --num-samples 10 \  
  --output-dir ./ryzen_test_output
```

Writes ryzen_test_output/reference/*.png, ryzen_test_output/search/*.png, and ryzen_test_output/manifest.csv with ground truth. Takes roughly the same time as on any other machine — expect a few seconds per sample.

5. Batch localization over that dataset

```
python localize.py \  
  --checkpoint checkpoints/m3_hn_r24/best.pt \  
  --manifest ryzen_test_output/manifest.csv \  
  --output ryzen_predictions.csv
```

Writes ryzen_predictions.csv with id, predicted_x, predicted_y, confidence, runtime_ms per pair — the runtime_ms column is exactly what you want to look at to see real CPU inference timing on this machine, per the hackathon spec's runtime-reporting requirement.

6. Validation report (small sample count)

```
python scripts/validation_report.py \
  --checkpoint checkpoints/m3_hn_r24/best.pt \
  --n-per-condition 10 \
  --out-dir ryzen_results
```

Runs 10 pairs under each of the four noise x geometry conditions (40 total) and writes `ryzen_results/validation_report.md`, `validation_report.json`, and `failure_case.png`. The JSON/Markdown report explicitly records hardware (via `platform.processor()` on CPU — will show your Ryzen model string), `python_version`, and the exact timing method used, satisfying the spec's "runtime per image pair, with hardware, Python version and timing method" requirement automatically.

7. Optional: tiny training smoke test (NOT real training)

NOTE This is for confirming the training code path runs end-to-end on this machine, nothing more. A model "trained" for 4 steps has learned nothing — do not evaluate its accuracy or draw any conclusions from it.

```
python scripts/train.py --run-name ryzen_smoke_test \
  --max-steps 4 --steps-this-run 4 \
  --val-every 2 --val-batches 1 --num-workers 0
```

`--num-workers 0` avoids spawning extra data-loading processes, which matters more on a CPU machine where every core is already doing useful work for the main process. If your Ryzen chip has many cores (8+) and you want to try a slightly larger smoke test, raise `--num-workers` to roughly half your physical core count instead of 0.

8. Command summary

Purpose	Command	Rough CPU time
Verify install	<code>pytest tests/ -q -m "not slow"</code>	40-90 s
Single-pair predict	<code>scripts/predict.py</code>	2-10 s
Generate 10-sample dataset	<code>generate_dataset.py</code>	~30-60 s
Batch localize (10 pairs)	<code>localize.py --manifest ...</code>	~20-100 s
Validation report (40 samples)	<code>scripts/validation_report.py --n-per-condition 10</code>	~2-6 min
Training smoke test (4 steps)	<code>scripts/train.py --max-steps 4 ...</code>	~1-3 min

These are rough, unverified estimates based on typical desktop-class CPU throughput relative to the GPU timings observed during this package's own development, not a benchmark measured on an actual Ryzen chip. Your real numbers will vary by core count, clock speed, and how many other processes are competing for CPU time — the validation report's own

runtime_ms output (Section 6) is the authoritative measurement for your specific machine.

9. Troubleshooting

Everything feels slow

Expected — see Section 1. Reduce sample counts (`--num-samples`, `--n-per-condition`) rather than waiting out a large run.

"could not find a version that satisfies torch==2.13.0+cu130"

You tried to install directly from `requirements.txt`. Use the plain `pip install torch torchvision` command in Section 2 instead — the pinned CUDA build in `requirements.txt` is only for the GPU setup path documented in the main README.

Out of memory / system feels unresponsive during training

Lower `--batch-size` (default 8) and/or `--num-workers` — each data-loading worker holds its own copy of in-flight canvas data, which adds up on a memory-constrained machine.